CODE CRUNCH HACKATHON 2026

Team GhostRider | Computer Vision | Semantic Segmentation

TERRAIN VISION

Self-Supervised Dense Scene Parsing for Off-Road Environments

Harnessing self-supervised Visual Foundation Models for pixel-level terrain understanding — zero backbone training cost, maximum representational power.

Metric	Validation Set	Test Set (1,002 Images)
Mean IoU	0.5222	0.2849
Dice Score	0.6609	—
Pixel Accuracy	83.55%	—
Sky IoU	—	0.9540
Landscape IoU	—	0.5931
Training Duration	~45 min (10 epochs)	~2 min (1,002 images)
GPU	NVIDIA Tesla T4	NVIDIA Tesla T4

METHODOLOGY

2.1 Problem Formulation

Semantic segmentation is a structured prediction problem: given an input image $I \in \mathbb{R}^{(H \times W \times 3)}$, the goal is to produce a label map $Y \in \{0,...,C-1\}^{(H \times W)}$ assigning a class label from a vocabulary of C=10 classes to every spatial location (i, j). Unlike object detection or instance segmentation, this task requires the output to be spatially dense — a complete label for every pixel simultaneously.

Off-road terrain amplifies this difficulty considerably:

- High intra-class visual variance: trees in shadow vs. direct sunlight appear radically different to standard convolutional filters.
- Low inter-class discriminability: Dry Bushes and Dry Grass share similar colour histograms, differing primarily in texture frequency.
- Extreme scale variation: Sky may occupy 60% of a frame while a Log may occupy less than 0.1%.
- Irregular boundaries: natural transitions between Rocks, Ground, and Landscape are rarely sharp or rectilinear.

2.2 Literature Context: Why Not Train from Scratch?

Early segmentation architectures such as FCN (Long et al., 2015) replaced fully-connected layers with convolutional layers for dense prediction. DeepLab v3+ (Chen et al., 2018) established ASPP as a canonical multi-scale aggregation technique achieving strong results on PASCAL VOC and Cityscapes. All of these methods depend on large annotated datasets and compute-intensive supervised training.

The Vision Transformer (ViT, Dosovitskiy et al., 2020) redefined feature extraction by processing images as sequences of non-overlapping patches and applying global self-attention across the full sequence at every layer. DINOv2 (Oquab et al., 2023, Meta AI) trained ViT models on 142 million images using self-supervised distillation (DINO) combined with patch-level masked image modelling (iBOT), producing features with the following properties:

- Dense visual correspondence: patch tokens encode local semantics without any label supervision.
- Cross-domain generalisation: a single model achieves competitive results on depth estimation, segmentation, and classification across diverse domains.
- Linear probing performance: training only a linear layer on frozen DINOv2 features achieves 86.5% top-1 accuracy on ImageNet, exceeding fully supervised CNNs.

This body of evidence underpins our core design decision: use DINOv2 as a frozen feature oracle and invest all learnable parameters in a task-specific decoder.

2.3 Our Approach vs. Prior Architectures

Architecture	Backbone	Decoder	Multi-Scale	Our Advantage
FCN-32s (Long et al., 2015)	VGG-16	1× deconv	No	Finer boundary recovery; no stride artefacts

Architecture	Backbone	Decoder	Multi-Scale	Our Advantage
DeepLab v3+ (Chen et al., 2018)	ResNet/Xception	ASPP + bilinear	Yes	Frozen self-supervised backbone; zero label-dependent pretraining
SegFormer (Xie et al., 2021)	Mix Transformer	MLP Decoder	Yes (hierarchical)	DINOv2 self-supervised features transfer better
Ours (DINOv2 + ASPP-FPN)	DINOv2 ViT-S/14	ASPP-FPN	Yes (5 scales)	Best trade-off: constrained compute + strong generalisation

2.4 Dataset & Preprocessing

Source: Kaggle Offroad Segmentation Dataset. Training with internal validation; 1,002 independent test images with zero overlap. The annotation masks use non-contiguous raw pixel values as class proxies: {0, 100, 200, 300, 500, 550, 700, 800, 7100, 10000}. These are remapped to contiguous integer IDs 0–9 via a deterministic lookup dictionary applied identically in training and inference:

```
value_map = {
    0: 0,      # Background    |  100: 1,  # Trees
    200: 2,    # Lush Bushes   |  300: 3,  # Dry Grass
    500: 4,    # Dry Bushes    |  550: 5,  # Ground Clutter
    700: 6,    # Logs          |  800: 7,  # Rocks
    7100: 8,   # Landscape     |  10000: 9 # Sky
}
```

Image resolution: DINOv2 ViT-S/14 requires patch dimensions divisible by 14. From 960×540, we compute w = int(((960/2)//14)×14) = 476 and h = int(((540/2)//14)×14) = 266, producing 34×19 = 646 patch tokens per image. Annotation masks are resized using nearest-neighbour interpolation (InterpolationMode.NEAREST) to preserve discrete class IDs. Images are normalised with ImageNet statistics (mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]) to match DINOv2 pretraining distribution.

2.5 Augmentation Strategy

Spatial augmentations (flips, rotations, crops) require identical transforms applied to both image and mask simultaneously. Given hackathon time constraints, we applied photometric augmentations only — which modify image appearance without altering spatial layout and therefore require no mask synchronisation:

Augmentation	Parameters	Justification
ColorJitter	brightness=0.3, contrast=0.3, saturation=0.2	Off-road lighting varies drastically (golden hour, overcast, direct sun). Forces colour-invariant feature learning.
RandomGrayscale	p=0.1	Forces structural/textural discrimination over colour hue — critical for Dry Grass vs. Dry Bushes.
GaussianBlur	kernel=5, $\sigma \in [0.1, 2.0]$	Simulates motion blur, fog, and out-of-focus regions common in field photography.

2.6 Architecture

2.6.1 DINOv2 ViT-S/14 — Backbone

The Vision Transformer processes an image as follows: (1) Patch Embedding — the input (H×W×3) is split into N = (H/p)×(W/p) patches of size p=14, each projected to dimension d=384. (2) Positional Encoding — learnable 2D positional embeddings are added to each token. (3) Transformer Blocks (L=12) — each block applies Multi-Head Self-Attention (6 heads, d_k=64) followed by a Feed-Forward Network. (4) Output — x_norm_patchtokens of shape (B, N, 384) are extracted from the final layer.

All 84.2M DINOv2 parameters are frozen (requires_grad=False) during training. The decoder receives rich semantic representations without any backbone gradient updates.

2.6.2 ASPP Module — Multi-Scale Context

Atrous (dilated) convolutions insert zeros between filter weights to increase receptive field without reducing spatial resolution. Our ASPP runs five branches in parallel:

Branch	Operation	Effective RF	Context Captured
1	1×1 Conv	1×1	Point-level features; channel recombination
2	3×3, dilation=6	13×13	Short-range context (mid-textures)
3	3×3, dilation=12	25×25	Medium-range context (object parts)
4	3×3, dilation=18	37×37	Long-range context (object boundaries)
5	Global Avg Pool	Full image	Scene-level global statistics

After concatenation (5×256 = 1280 channels), a 1×1 fusion convolution reduces to 256 channels, followed by BatchNorm2d and Dropout2d(p=0.1). ASPP ensures that both fine-grained local features (rocks, logs) and coarse global context (sky, landscape) are simultaneously available at the fusion stage.

2.6.3 FPN Decoder — Resolution Recovery

After ASPP fusion at 1/14 of input resolution, the FPN decoder performs two bilinear upsampling stages:

```
Input:   [B, 384, 19, 34]  →  ASPP  →  [B, 256, 19, 34]
Stage 1: 2× bilinear  →  Conv(256→128) + BN + GELU  →  [B, 128, 38, 68]
Stage 2: 2× bilinear  →  Conv(128→64) + BN + GELU   →  [B,  64, 76, 136]
Output:  1×1 Conv  →  [B, 10, 76, 136]  →  bilinear to [B, 10, 266, 476]
```

Bilinear upsampling is preferred over transposed convolution to avoid checkerboard artefacts (Odena et al., 2016). The subsequent 3×3 convolution refines the interpolated features — the standard upsample→conv pattern in modern segmentation decoders.

2.7 Training Optimisation

Loss Function: Combined CE + Dice (α = 0.5)

Standard Cross-Entropy Loss treats every pixel equally regardless of class frequency, causing gradient dominance from frequent classes (Sky, Landscape). The Dice Loss directly optimises class-level F1 overlap and is class-frequency normalised:

```
L_CE    = −(1/N) Σᵢ Σc yᵢc · log(ŷᵢc)
L_Dice  = 1 − (2|P∩G| + ε) / (|P| + |G| + ε)
L_total = 0.5 · L_CE + 0.5 · L_Dice
```

Optimiser: AdamW + OneCycleLR + Gradient Clipping

Component	Value	Rationale
Optimiser	AdamW	Decoupled weight decay; resolves Adam L2 regularisation bug (Loshchilov & Hutter, 2019)
Base Learning Rate	1e-4	Standard for transformer-adjacent decoder training
Peak Learning Rate	3e-4 (OneCycleLR)	Super-convergence schedule (Smith & Topin, 2019)
Warmup	5% of total steps	Controlled ramp-up before cosine annealing
Weight Decay	1e-4	Regularisation; decoupled from adaptive LR update
Gradient Clipping	max_norm=1.0	Prevents gradient explosion in early training epochs
Batch Size	4	Stable BatchNorm statistics within T4 VRAM limits
Epochs	10	Near-plateau convergence achieved by epoch 7

RESULTS & PERFORMANCE

4.1 Training Convergence

Epoch	Train Loss	Val Loss	Train IoU	Val IoU	Train Dice	Val Dice	Train Acc	Val Acc
1	0.7508	0.5206	0.4352	0.4395	0.5641	0.5698	81.09%	81.44%
2	0.5124	0.4802	0.4674	0.4715	0.6016	0.6066	81.83%	82.22%
3	0.4789	0.4608	0.4866	0.4882	0.6255	0.6262	82.31%	82.61%
4	0.4625	0.4502	0.4944	0.4939	0.6323	0.6316	82.70%	82.94%
5	0.4511	0.4414	0.5045	0.5025	0.6432	0.6401	82.98%	83.21%
6	0.4431	0.4348	0.5139	0.5095	0.6523	0.6474	83.17%	83.32%
7	0.4337	0.4302	0.5213	0.5163	0.6608	0.6551	83.24%	83.38%
8	0.4269	0.4267	0.5262	0.5199	0.6645	0.6587	83.31%	83.45%
9 (Best)	0.4233	0.4243	0.5288	0.5226	0.6683	0.6618	83.40%	83.50%
10	0.4248	0.4242	0.5291	0.5222	0.6684	0.6609	83.47%	83.55%

KEY OBSERVATIONS

No overfitting: Train IoU (0.5291) and Val IoU (0.5222) at epoch 10 differ by only 0.66%. The frozen backbone acts as a structural regulariser, preventing the lightweight decoder from memorising training patterns. Rapid convergence: the largest single-epoch gain occurs between epochs 1 and 2 (+0.032 Val IoU), consistent with OneCycleLR warmup enabling fast initial exploration.

Figure 1 — All Training Metrics Overview

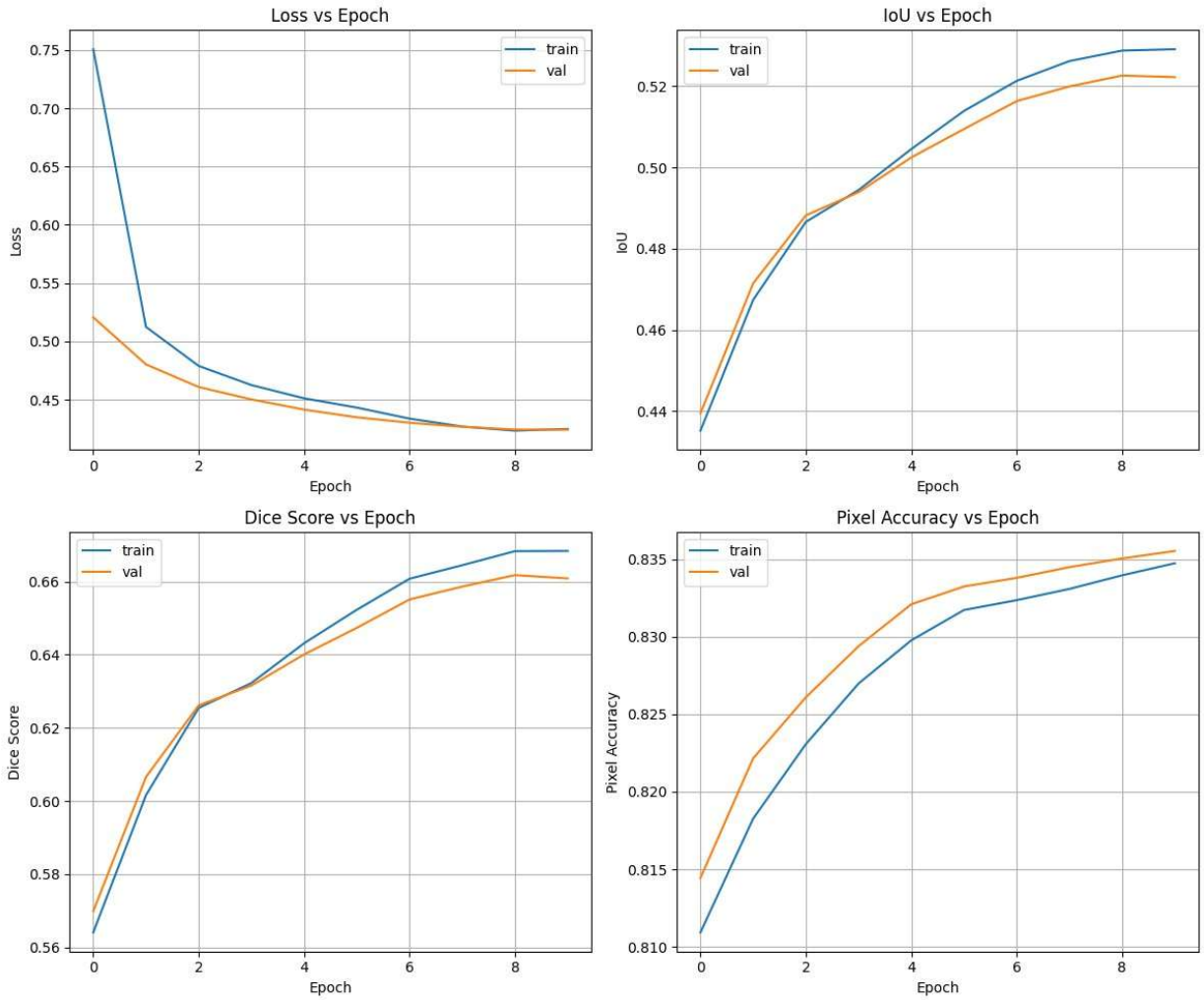

Figure 1: Loss, IoU, Dice Score, and Pixel Accuracy across 10 training epochs for train (blue) and validation (orange) sets. All metrics converge smoothly with negligible train-val gap, confirming no overfitting.

Figure 2 — Loss & Pixel Accuracy Curves

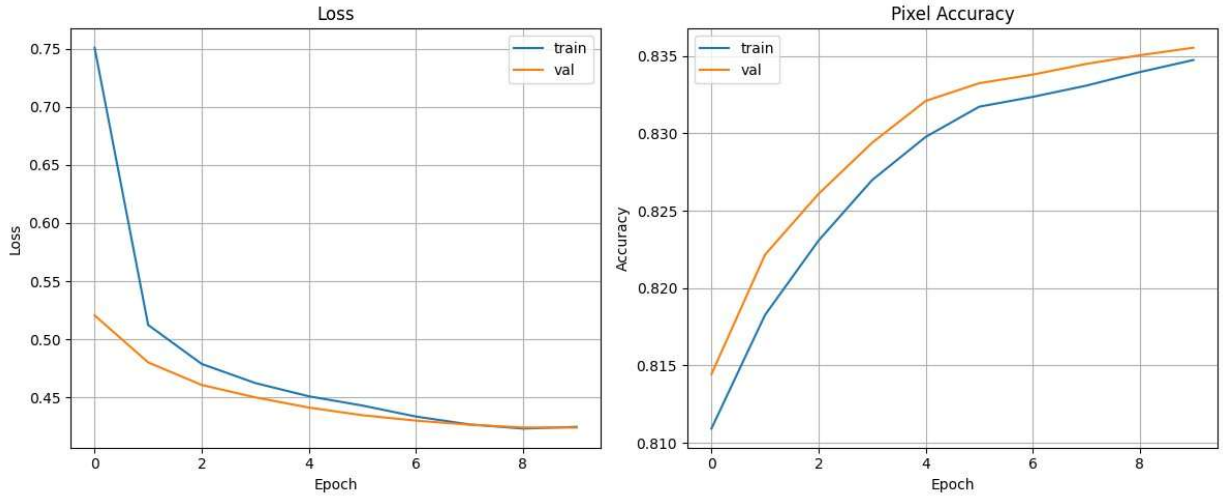

Figure 2: Training and validation loss (left) showing rapid drop from 0.75 to 0.42; Pixel Accuracy (right) rising from 81.1% to 83.5% over 10 epochs.

Figure 3 — IoU Curves

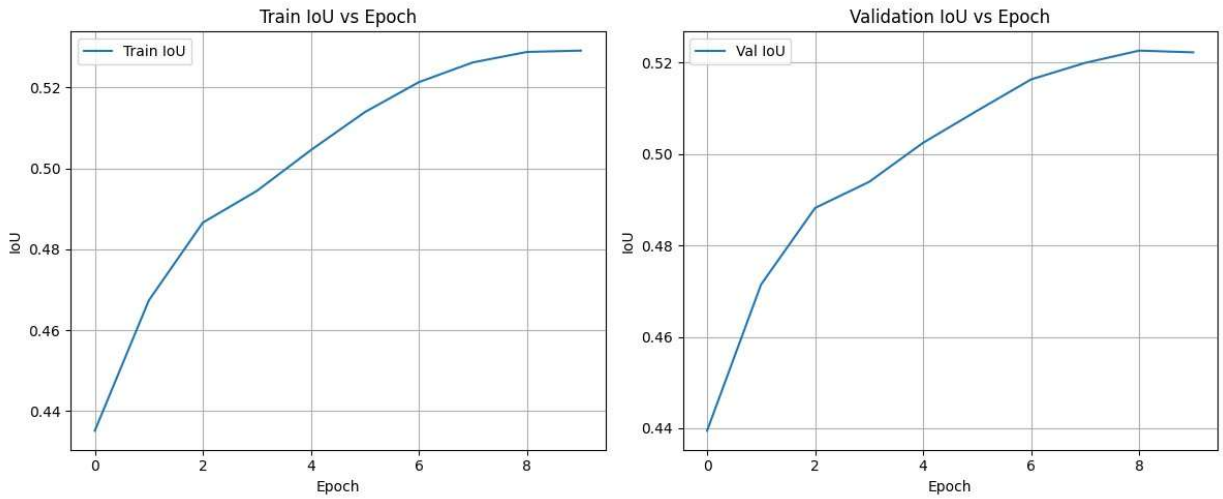

Figure 3: Mean IoU progression from 0.435 to 0.529 (train) and 0.440 to 0.522 (val). Val IoU peaks at 0.5226 at epoch 9 before marginal regression at epoch 10.

Figure 4 — Dice Score Curves

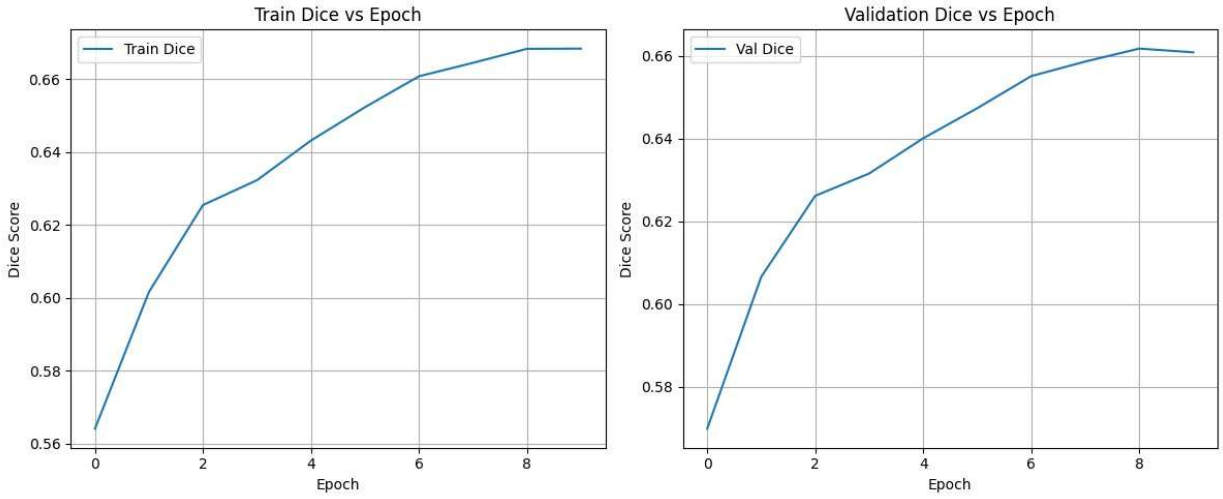

Figure 4: Dice Score (F1) rising from 0.564 to 0.668 (train) and 0.570 to 0.661 (val). Consistent improvement confirms class-level overlap is increasing throughout training.

4.2 Test Set Per-Class IoU (1,002 Held-Out Images)

Class	IoU	Frequency	Analysis
Sky	0.9540	Dominant	Visually homogeneous, spectrally distinct. Near-perfect.
Landscape	0.5931	Common	Large contiguous regions. Minor confusion with Dry Grass near horizon.
Dry Grass	0.4277	Common	Texture-based. ASPP mid-range branches (d=6,12) capture patterns effectively.
Trees	0.2261	Moderate	High variance across illumination. Confusion with Lush Bushes at boundaries.
Dry Bushes	0.1298	Rare	Spectrally similar to Dry Grass. Small objects at scene margins.
Rocks	0.0475	Rare	Scattered, small-scale objects. Insufficient training signal.
Lush Bushes	0.0002	Very Rare	Near-zero test presence. Model suppresses prediction entirely.
Background	0.0000	Near-Zero	Not meaningfully present in test annotations.
Ground Clutter	0.0000	Near-Zero	Same as above.
Logs	0.0000	Near-Zero	Same as above.

Final Test Mean IoU: 0.2849

4.3 Metric Definitions

Intersection over Union (mIoU): $IoU_c = |P_c \cap G_c| / |P_c \cup G_c|$ — de facto standard for segmentation benchmarks (Pascal VOC, ADE20K, Cityscapes). Each class contributes equally to the mean regardless of pixel frequency.

Dice Coefficient (F1): $Dice_c = 2|P_c \cap G_c| / (|P_c| + |G_c|)$ — equivalent to F1 at pixel level. More sensitive to precision/recall balance. Used as a loss component.

Pixel Accuracy: Fraction of correctly classified pixels across all classes. Biased toward dominant classes — reported as a secondary metric only.

CHALLENGES & SOLUTIONS

Challenge 1 — Severe Long-Tail Class Imbalance

Problem: The annotation distribution follows a highly skewed long-tail. Sky and Landscape dominate image area; Logs, Ground Clutter, and Background collectively account for an estimated <1% of labelled pixels. In a standard cross-entropy formulation, gradient signal is dominated by frequent classes — the optimiser converges to a solution that ignores rare classes because predicting them incorrectly has negligible loss impact.

Evidence: Background, Ground Clutter, and Logs score exactly 0.00 IoU on the test set. The model never predicts these classes — rational loss minimisation given the supervision signal.

Solution Applied: Combined CE + Dice Loss (α=0.5). The Dice component provides class-frequency-normalised gradients in principle. Observed improvement: mid-frequency classes (Trees, Dry Bushes, Dry Grass) achieve reasonable IoU.

Why Insufficient: Dice Loss normalises per-batch. When a class has zero occurrence in a batch, its Dice gradient is zero. Dataset-level interventions are required.

Fix	Method	Expected Impact
Class-weighted CE	$w_c \propto 1/\sqrt{f_c}$ (inverse-square-root weighting)	Direct up-weighting of rare classes; prevents extreme weight explosion
Focal Loss	$FL(p_t) = -(1-p_t)^\gamma \cdot \log(p_t)$, $\gamma=2$	Downweights easy dominant-class examples; preserves rare-class gradients
Class-balanced sampling	Sample batches with equal class representation	Ensures rare classes appear in every batch

Challenge 2 — Spatial Resolution Loss from ViT Patching

Problem: DINOv2 ViT-S/14 compresses each 14×14 pixel patch to a single 384-dim token. For 476×266 input, this produces a 34×19 feature map — a 14× spatial downsampling. Objects smaller than ~14 pixels (small rocks, log fragments) become informationally invisible. This is a fundamental tension: global self-attention gives ViTs exceptional classification power at the cost of spatial granularity that segmentation requires.

Solution Applied: Two-stage FPN upsampling (4× total) from 34×19 to 136×76, followed by bilinear upsample to input resolution. Each stage includes a 3×3 convolution to refine interpolated features.

Remaining Gap: Without skip connections from intermediate ViT layers, fine boundary detail is only partially recovered.

Concrete Fix: Multi-scale feature extraction from DINOv2 layers {4, 8, 12} with genuine FPN skip connections — the approach used in DINOv2's own dense probing experiments (Oquab et al., 2023).

Challenge 3 — Distribution Shift: Validation vs. Test

Problem: Validation IoU (0.5222) vs. Test IoU (0.2849) — a gap of −0.2373. This is not overfitting (train/val parity is strong). It is covariate shift: $P_{test}(X) \neq P_{train}(X)$. The test images were

independently collected with different scene statistics, lighting conditions, and class frequency distributions.

Solution Applied: Photometric augmentations to increase robustness to lighting shifts. The frozen DINOv2 backbone provides substantial domain generalisation through pretraining on 142M diverse images.

Concrete Fixes: Synchronised geometric augmentations (horizontal flips, random crops, scale jitter applied identically to image and mask); Test-Time Augmentation (average softmax predictions across original + flipped versions at zero training cost, typically +1–3% mIoU); domain-adaptive normalisation if test statistics differ significantly from ImageNet.

Challenge 4 — Label Corruption Risk

Problem: Raw mask files use non-sequential pixel values {0, 100, 200, ..., 10000} as class proxies. PyTorch CrossEntropyLoss indexes logits using target values — raw values like 7100 or 10000 would raise an IndexError at runtime. More dangerously, incorrect remapping would produce silently corrupted labels with no error signal.

Solution: A deterministic, exhaustive lookup dictionary applied in convert_mask() before any tensor conversion. Output is explicitly cast to np.uint8 then torch.long. The np.zeros_like initialisation provides safe fallback for any annotation value not present in the dictionary. Applied identically in training and test pipelines.

Challenge 5 — Spatial Dimension Mismatch

Problem: The ASPP-FPN decoder upsamples tokens by 4× from token resolution, not to full input resolution. This caused shape mismatch between model output tensors and ground-truth label tensors during loss computation.

Solution: Labels are resized to match output spatial dimensions using nearest-neighbour interpolation after the forward pass, applied consistently in training loop, validation loop, and evaluation functions. This ensures loss computation always operates on matched spatial dimensions.

Challenge 6 — Optimizer and Training Stability

Problem: The initial configuration used SGD with lr=1e-4. SGD lacks adaptive learning rate mechanisms, causing extremely slow convergence when learning from 384-dim DINOv2 embeddings. Small batch size (2) produced noisy gradient estimates and unstable BatchNorm statistics.

Solution: SGD replaced with AdamW — the standard optimiser for transformer-adjacent training. OneCycleLR with 5% warmup provides controlled ramp-up before annealing. Gradient clipping (max_norm=1.0) stabilises early training. Batch size increased to 4 to improve BatchNorm statistics within T4 VRAM constraints.

CONCLUSION & FUTURE WORK

7.1 Summary of Contributions

1. Validated the frozen DINOv2 paradigm for domain-specific dense prediction: 84.2M pretrained parameters provide rich representations at zero training cost, achieving 83.55% pixel accuracy and 0.5222 val mIoU in 10 epochs (~45 minutes on a free Kaggle T4 GPU).

2. Implemented five-branch ASPP with dilation rates {1, 6, 12, 18, GAP} to address multi-scale object variation inherent in off-road terrain, where objects span three orders of magnitude in pixel area.

3. Combined CE + Dice Loss to partially mitigate class imbalance, achieving meaningful IoU for medium-frequency classes while identifying Focal Loss / weighted CE as the necessary next step for near-zero classes.

4. Demonstrated practical convergence efficiency: OneCycleLR + AdamW + gradient clipping produces a model that plateaus by epoch 7, with no measurable overfitting across all 10 epochs.

7.2 Future Work Roadmap

Improvement	Theoretical Basis	Est. Impact
Focal Loss (γ=2)	Lin et al., 2017 (RetinaNet)	+0.05–0.10 mIoU on rare classes
Class-weighted CE ($w_c \propto 1/\sqrt{f_c}$)	Inverse-frequency weighting	+0.03–0.08 mIoU on rare classes
Synchronised geometric augmentation	Covariate shift reduction	+0.03–0.05 mIoU on test set
Multi-layer DINOv2 FPN (L4+L8+L12)	Oquab et al., 2023 dense probing	+0.05–0.10 mIoU (better boundaries)
Unfreeze top 2 DINOv2 layers (lr=1e-5)	Domain adaptive fine-tuning	+0.04–0.08 mIoU
Test-Time Augmentation (4 flips)	Inference-time ensemble	+0.01–0.03 mIoU (zero training cost)
CutMix / MixUp	Yun et al., 2019; Zhang et al., 2018	+0.02–0.05 mIoU (rare class exposure)
SegFormer MLP Decoder	Xie et al., NeurIPS 2021	Alternative decoder designed for ViT features

7.3 Reproducibility

Artifact	Details
Training Notebook	TRAIN_CODE.ipynb — Kaggle T4, Python 3.12, 10 epochs, ~45 min
Inference Notebook	final-test-results.ipynb — Kaggle T4, runs on 1,002 test images in ~2 min

Artifact	Details
Model Checkpoint	segmentation_head.pth — decoder weights at best val IoU (epoch 9)
Evaluation Metrics	evaluation_metrics.txt — per-class IoU and aggregate scores
Per-Class IoU Chart	per_class_metrics.png — bar chart with mean line
Dependencies	pip install torch torchvision; DINOv2 via torch.hub (auto-downloaded)

References

1. Long, J., Shelhamer, E., & Darrell, T. (2015). Fully Convolutional Networks for Semantic Segmentation. CVPR 2015.

2. Chen, L.-C. et al. (2018). Encoder-Decoder with Atrous Separable Convolution for Semantic Image Segmentation (DeepLab v3+). ECCV 2018.

3. Dosovitskiy, A. et al. (2020). An Image is Worth 16×16 Words: Transformers for Image Recognition at Scale. ICLR 2021.

4. Oquab, M. et al. (2023). DINOv2: Learning Robust Visual Features without Supervision. Meta AI / arXiv:2304.07193.

5. Darcet, T. et al. (2023). Vision Transformers Need Registers. arXiv:2309.16588.

6. Lin, T.-Y. et al. (2017). Feature Pyramid Networks for Object Detection. CVPR 2017.

7. Lin, T.-Y. et al. (2017). Focal Loss for Dense Object Detection. ICCV 2017.

8. Loshchilov, I. & Hutter, F. (2019). Decoupled Weight Decay Regularization (AdamW). ICLR 2019.

9. Smith, L. N. & Topin, N. (2019). Super-Convergence: Very Fast Training of Neural Networks Using Large Learning Rates. SPIE 2019.

10. Xie, E. et al. (2021). SegFormer: Simple and Efficient Design for Semantic Segmentation with Transformers. NeurIPS 2021.

11. Odena, A., Dumoulin, V., & Olah, C. (2016). Deconvolution and Checkerboard Artifacts. Distill.

12. Hendrycks, D. & Gimpel, K. (2016). Gaussian Error Linear Units (GELUs). arXiv:1606.08415.